Capitolo 1

Algoritmi su Grafi

1.1 Cammini Minimi da Sorgente Singola

Esercizio 1.1.1 #DagShortestPaths #Ottimalità

Dato un grafo $G = (V, E)$ diretto aciclico e pesato $w : E \to \mathbb{N}$, dato un intero $k \in \mathbb{N}$, e data una sorgente $s \in V$, trovare i vertici in V che distano al più k da s.
Descrivere un algoritmo che risolve il problema e darne lo pseudocodice.
Studiare la complessità dell'algoritmo proposto.
Discutere l'ottimalità della soluzione proposta.

Elaborazione Soluzione: Il grafo è pesato. I pesi non hanno limitazioni. Per calcolare gli shortest paths, possiamo applicare l'algoritmo di Bellman Ford o quello di Floyd-Warshall. Poiché sappiamo che il grafo è diretto ed aciclico, possiamo eseguire la varante di Bellman Ford che considera gli archi una sola volta (e non $|V|$). Chiamiamo tale variante Dag-Shortest-Paths. L'algoritmo assume di rilassare i vertici nell'ordine dato dal sort-topologico del grafo. Dato che a noi interessano i vertici a distanza al più k, una volta che l'algoritmo ha finito di trovare le distanze minime dalla sorgente scorriamo tutti i nodi e selezioniamo quelli a distanza minore od uguale a k.

Pseudocodice:

Dag-Shortest-Paths-Dist-k(**Grafo** G, **Funzione Peso** w, **Sorgente** s, **int** k)
Lista $sol = \emptyset$ // Qui va la soluzione
Lista $L =$ TOPOLOGICAL-SORT(G) // DFS dove ordinano nodi per tempo fine visita
// L'inizializzazione:
for *ogni vertice* $v \in G.V$ **do**
  $v.d = +\infty$
  $v.predecessore =$ NULL
$s.d = 0$
for *ogni vertice* $u \in G.V$ *presi secondo l'ordine topologico* **do**
  for *ogni vertice* $v \in G.Adj[u]$ **do**
    // Il rilassamento:
    if $v.d > u.d + w(u, v)$ **then**
      $v.d = u.d + w(u, v)$
      $v.predecessore = u$
// cerchiamo quelli di nostro interesse
for *ogni vertice* $u \in G.V$ **do**
  if $v.d \leq k$ **then**
    INSERT(v, sol)
return: *sol*

Complessità: L'algoritmo DAG-SHORTEST-PATHS-DIST-K sfrutta il fatto che il grafo sia un DAG, infatti il sort topologico permette di fare la relax una sola volta per ogni arco; questo perché l'ordinamento topologico ci fornisce un cammino semplice che va da un nodo a tutti i suoi successivi.

Il costo di DAG-SHORTEST-PATHS-DIST-K è $O(V + E)$, che è ottimo; infatti il grafo rappresentato con liste di adiacenza ha dimensione $(V + E)$; lo scorrere tutti i vertici per trovare quelli a distanza al più k ci costa $O(V)$, quindi il costo resta asintoticamente $O(V + E)$.

Esercizio 1.1.2 #Dijkstra

Dato un grafo pesato, con pesi non negativi, ed una destinazione s, e data una costante k, trovare i vertici a distanza $\leq k$ dalla destinazione s.

Elaborazione Soluzione: Notiamo che il problema dice *destinazione* s, ed inoltre pone un limite k alla distanza dei vertici dalla destinazione. Per poter usare Dijkstra a noi serve una sorgente, quindi dobbiamo invertire tutti gli archi del grafo, quindi dobbiamo trovare il Grafo Trasposto di G, chiamato G'; a questo punto possiamo eseguire Dijkstra su G' impedendogli di proseguire oltre la distanza k.

Pseudocodice: Supponiamo che il grafo sia rappresentato con una lista di adiacenza.

```
GrafoTrasposto(Grafo G)
Grafo G' // Il nuovo grafo avrà E, V ed adj[]
G'.V = G.V
for ogni arco (u, v) ∈ G.E do
    G'.E = (v, u)
for ogni vertice u ∈ G.V do
    for ogni vertice v ∈ G.adj[u] do
        APPEND(G'.adj[v], u) // Mettiamo u nella lista di adiacenza di v
return: G'

Dijkstra_K(Grafo G, Funzione Peso ω, Sorgente S)
for ogni vertice v ∈ G.V do
    v.d = +∞
    v.predecessore = NULL
S.d = 0
Lista L = ∅
Min-Heap Q = ∅
for ogni vertice v ∈ G.V do
    INSERT(Q, v)
stop = FALSE // Variabile per fermare il ciclo while
while IS-EMPTY(Q) == FALSE and stop == TRUE do
    u = EXTRACTMIN(Q)
    // Rilassiamo gli archi connessi ad u fino a stima k
    if u.d > k then
        stop = TRUE
        // Usciamo dal ciclo appena troviamo un vertice a distanza maggiore di k
    else
        for ogni vertice v ∈ G.adj[u] do
            if v.d > u.d + ω(u, v) then
                v.d = u.d + ω(u, v)
                v.predecessore = u
    ENQUEUE(L, u)
```

Funzionamento:

Al fine di risolvere la richiesta dell'esercizio, abbiamo combinato la funzione che trova il trasposto di un grafo con una versione modificata dell'algoritmo di Dijkstra. In particolare, la funzione GrafoTrasposto dato un grafo G calcola e restituisce il suo trasposto G'. Poiché i vertici rimangono invariati da un grafo al suo trasposto, la funzione assegna $G'.V = G.V$. Per trovare invece l'insieme di archi $G'.E$ abbiamo invertito tutte le coppie di vertici in $G.E$. Infine, per trovare la lista di adiacenza trasposta, abbiamo letto tutta la lista di adiacenza di G, e la abbiamo trascritta in $adj'[]$ invertendo le connessioni dei vertici.

Per quanto riguarda Dijkstra_K, abbiamo aggiunto una condizione di stop all'algoritmo di Dijkstra originale per fare in modo che si fermi al primo vertice che estrae a distanza superiore a k; infatti, poiché l'algoritmo di Dijkstra restituisce i vertici con distanza crescente dalla sorgente, che in questo caso è la destinazione.

Quindi, per prima cosa, abbiamo trovato calcolato G': V, E, $adj[]$, dopodiché abbiamo aggiunto dei controlli per fermare l'algoritmo quando la variabile booleana *stop*, inizializzata a False, diviene True, evento che si verifica quando si estrae dal min-heap un vertice avente stima del cammino minimo maggiore del nostro limite k.

Complessità: Il costo dell'implementazione con il min-heap in DIJKSTRA_K è:

$$T(n) = T(|V| + |E|)^{\text{GRAFOTRASPOSTO}} + T(|V| + |E|)^{\text{DIJKSTRA_K}} =$$

$$= O(|V| + |E|) + O((|V| + |E|) \log_2 |V|) = O((|V| + |E|) \log_2 |V|)$$

Infatti:

- $T(n)^{\text{GRAFOTRASPOSTO}} = O(V + E)$: Infatti per realizzare il grafo trasposto dobbiamo fare $O(E)$ operazioni per trovare $G'.E$, poiché dobbiamo leggere tutto $G.E$ ed $O(V + E)$ operazioni per trovare $G'.adj[]$, poiché dobbiamo leggere tutta la lista $G.adj[]$; per un costo finale di $O(V + E)$.
- $T(n)^{\text{DIJKSTRA_K}} = O((V_k + E_k) \log_2 V_k)$: Infatti è il costo dell'algoritmo di Dijkstra implementato con min-heap, dove V_k è l'insieme dei vertici che si qualificano per l'estrazione (ossia con distanza $\leq k$ dalla destinazione) ed $E_k \in V_k^2$.

Esercizio 1.1.3 #DagShortestPaths #Ottimalità

Dato un grafo orientato e aciclico $G = (V, E)$ pesato $w : E \to \mathbb{R}$ e data una sorgente $s \in V$, progettare un algoritmo per trovare i cammini di costo minimo da s verso tutti gli altri vertici in V.
Discutere la soluzione proposta e discutere la complessità in tempo della soluzione proposta.
La soluzione proposta è ottima? Motivare la risposta.

Elaborazione Soluzione: Eseguiamo l'algoritmo DAG-SHORTEST-PATHS che effettua un sort-topologico per poter fare la relax una sola volta di tutti gli archi seguendo l'ordinamento topologico.

Pseudocodice:

Dag-Shortest-Paths(**Grafo** G, **Funzione Peso** ω, **Sorgente** s)
List L = TOPOLOGICAL-SORT(G) // DFS collezionando nodi per tempo fine visita
// L'inizializzazione:
for *ogni vertice* $v \in G.V$ **do**
  $v.d = +\infty$
  $v.predecessore$ = NULL
$s.d = 0$
for *ogni vertice* $u \in G.V$ *presi secondo l'ordine topologico in* L **do**
  for *ogni vertice* $v \in G.Adj[u]$ **do**
    // Il rilassamento:
    if $v.d > u.d + \omega(u, v)$ **then**
      $v.d = u.d + \omega(u, v)$
      $v.predecessore = u$

Complessità: Il costo di DAG-SHORTEST-PATHS è $O(V + E)$ che è ottimo; infatti il grafo rappresentato con liste di adiacenza ha dimensione $(V + E)$.

Esercizio 1.1.4 #Bellman-Ford

Dato un grafo $G = (V, E)$, orientato e pesato con pesi arbitrari, trovare se esiste un ciclo di costo negativo raggiungibile da una sorgente s.

- Descrivere l'algoritmo che si propone
- Darne la descrizione in pseudocodice
- Discuterne la complessità in tempo dell'algoritmo.

Elaborazione Soluzione: Possiamo utilizzare l'algoritmo di Bellman-Ford. Esso calcola i cammini minimi da sorgente singola per tutti i nodi raggiungibili.

Pseudocodice:

```
Bellman-Ford(Grafo G, Funzione Peso ω, Sorgente s)
// L'inizializzazione:
for ogni vertice v ∈ G.V do
    v.d = +∞
    v.predecessore = NULL
s.d = 0
for i = 1 to |G.V| − 1 do
    for ogni arco (u, v) ∈ G.E do
        // Il rilassamento:
        if v.d > u.d + ω(u, v) then
            v.d = u.d + ω(u, v)
            v.predecessore = u
for ogni arco (u, v) ∈ G.E do
    if v.d > u.d + ω(u, v) then
        return: FALSE
return: TRUE
```

Complessità: Bellman-Ford implementato con liste di adiacenza ha costo $O(V \cdot E)$ dato dal rilassare tutti gli archi del grafo, $|E|$, per $|V|$ volte.

Esercizio 1.1.5 #Dijkstra

Dato un grafo $G = (V, E)$ diretto e pesato $\omega : E \to N^+$, data una sorgente $s \in V$, trovare il vertice u' tale che il cammino da s a u' sia quello di peso $d(s, u')$ minimo in G. Ossia $d(s, u') = \min_{u \in V} d(s, u)$.
Descrivere un algoritmo che risolve il problema e darne lo pseudocodice. Studiare la complessità dell'algoritmo proposto.

Elaborazione Soluzione: Abbiamo un grafo pesato con pesi non negativi dove vogliamo trovare il vertice più vicino alla sorgente. Possiamo eseguire l'algoritmo di Dijkstra per trovare le distanze di ogni vertice dalla sorgente e poi scorrerli tutti per trovare quello di nostro interesse.

Pseudocodice:

```
Dijkstra-Most-Close(Grafo G, Funzione Peso ω, Nodo Sorgente S)
// Inizializzazione
for ogni vertice v ∈ G.V do
    v.d = +∞
    v.predecessore = NULL
S.d = 0
Lista L = ∅
Min-Heap Q = ∅
for ogni vertice v ∈ G.V do
    INSERT(Q, v)
trovato = FALSE
while IS-EMPTY(Q) == FALSE and trovato == FALSE do
    u = EXTRACTMIN(Q)
    for ogni vertice v ∈ G.adj[u] do
        if v.d > u.d + ω(u, v) then
            v.d = u.d + ω(u, v)
            v.predecessore = u
    ENQUEUE(L, u)
Nodo n // Qui mettiamo la soluzione
int min = +∞ // Distanza minima
for ogni vertice v ∈ G.V do
    if v.d < min then
        min = v.d
        n = v
return: n
```

Complessità: L'algoritmo di Dijkstra implementato con un min-heap ha costo $O((E+V)\cdot \log V)$ che è maggiore del costo $O(V)$ usato per la ricerca del nostro nodo d'interesse. Poi si cerca u' cercando il vertice con distanza minima.

Attenzione: Nonostante l'algoritmo sia corretto, non sfrutta il fatto che i nodi ritornati da Dijkstra sono in ordine crescente di distanza dalla sorgente. Pertanto u' è il nodo in G con peso minimo uscente dalla sorgente! Quindi la complessità è $O(|Adj(s)|)$ ossia il grado del nodo sorgente.

Esercizio 1.1.6 #Dijkstra

Dato un grafo $G=(V,E)$ pesato $\omega : E \to R^+$, e dato l'insieme $F=\{u,w\}$ di due vertici speciali in V, assegnare ad ogni vertice $v \in V$ il vertice in F più vicino. Al solito la distanza fra due vertici è definita come il peso del cammino minimo fra i due vertici stessi.
Descrivere un algoritmo che risolve il problema e darne lo pseudocodice.

Elaborazione Soluzione: Per ogni nodo z dobbiamo calcolare $\delta(z,u)$ e $\delta(z,w)$, e ritornare il nodo u se $\delta(z,u) = \min\{\delta(z,u),\delta(z,w)\}$ o il nodo w se $\delta(z,w) = \min\{\delta(z,u),\delta(z,w)\}$. In modo stringato, si dice dobbiamo ritornare per ogni nodo z l'argomento c che minimizza $\min_{c\in u,w}\{\delta(z,c)\}$. Anziché calcolare per ogni nodo z gli shortest paths verso le destinazioni w e u. Come abbiamo già visto in un precedente esercizio, questo equivale ad eseguire l'algoritmo di Dijkstra sulla grafo trasposto avendo come sorgente u o w.

Pseudocodice:

Dijkstra(**Grafo** G, **Funzione Peso** ω, **Nodo Sorgente** S)
// Inizializzazione
for *ogni vertice* $v \in G.V$ **do**
  $v.d_S = +\infty$
  $v.predecessore =$ NULL
$S.d_S = 0$
Lista $L = \emptyset$
Min-Heap $Q = \emptyset$
for *ogni vertice* $v \in G.V$ **do**
  INSERT(Q, v)
while IS-EMPTY(Q) == FALSE **do**
  $u =$ EXTRACTMIN(Q)
  for *ogni vertice* $v \in G.adj[u]$ **do**
    if $v.d_S > u.d_S + \omega(u,v)$ **then**
      $v.d_S = u.d_S + \omega(u,v)$
      $v.predecessore = u$
  ENQUEUE(L, u)

Assign-F(**Grafo** G, **Funzione Peso** ω, **nodo** u, **nodo** w)
// Eseguiamo Dijkstra per ogni elemento in F
$G =$ TRANSPOSE(G) // Facciamo il trasposto
DIJKSTRA(G, ω, u)
DIJKSTRA(G, ω, w)
for *ogni vertice* $v \in G.V$ **do**
  if $v.d_u < v.d_w$ **then**
    $v.closest = u$
  else
    $v.closest = w$

Complessità: L'esecuzione dell'algoritmo di Dijkstra con min-heap costa $O((E+V)\cdot \log V)$; la lettura di tutti i vertici per l'assegnamento costa $O(V)$.

1.2 Cammini Minimi tra Tutte le Coppie

Esercizio 1.2.1 #MetodoTropicale

Dato un grafo $G = (V, E)$ connesso, pesato $w : E \to R$, ed orientato, data una sorgente $s \in V$, trovare il costo dei cammini minimi da s a tutti gli altri vertici in V di lunghezza al più 3. Dare lo pseudo-codice dell'algoritmo, calcolare la complessità in tempo dell'algoritmo proposto.

Elaborazione Soluzione: Abbiamo un grafo orientato e pesato con pesi qualsiasi (quindi potenzialmente anche negativi) avente una sorgente s dalla quale vogliamo trovare tutti i cammini minimi di lunghezza ≤ 3 verso ogni altro nodo del grafo; inoltre il grafo potrebbe avere dei cicli di costo negativo, visto che non è specificato il contrario; eventualmente individuabili con un esecuzione di Bellman-Ford di costo $O(V \cdot E)$.

Infatti per eseguire l'algoritmo sotto descritto si deve supporre che non vi siano cicli di costo negativo.

L'unico algoritmo che conosciamo per la ricerca di cammini minimi da sorgente singola nel contesto sopra descritto è Bellman-Ford. Ma Bellman-Ford non risolve il nostro problema, infatti i cammini dalla sorgente agli altri vertici che trova sono di lunghezza non specificata, mentre a noi servono quelli di lunghezza massima 3. Quindi ad esempio se tra la sorgente s ed il vertice v abbiamo un cammino di lunghezza 5, di costo 4, ed un cammino di lunghezza 3 con costo 7 troverà il primo, mentre a noi serve il secondo.

Dobbiamo sfruttare il Metodo Tropicale per la ricerca di cammini tra tutte le coppie di un grafo. Infatti questo algoritmo permette di trovare i cammini minimi tra tutte le coppie, di una certa lunghezza al più k, nella k-esima matrice da esso generata.

La soluzione al nostro problema sarà quindi nella terza matrice generata dall'algoritmo alla riga corrispondente al vertice s cioè la nostra sorgente.

Il Metodo Tropicale ha costo $O(n^4)$ ma poiché noi lo eseguiamo solo fino alla terza matrice e non fino alla n-esima, cioè quella matrice avente i cammini tra tutte le coppie di lunghezza al più n, il costo si riduce a $O(n^3)$.
Per ridurre ulteriormente il costo della nostra soluzione possiamo eseguire i calcoli non per l'intera matrice, ma solo per la riga corrispondente al vertice s; cioè l'unica di nostro interesse.

Pseudocodice:

Tropical-Single-Source(**Matrix** $W[n \text{ x } n]$) // W è la matrice di adiacenza con pesi
Array $R^1[n] = W[s]$ // Selezioniamo la riga corrispondente ad s
for $m = 2$ **to** 3 **do**
  // Ci serve il terzo vettore R^3
  $R^m = \text{EXTENDED-SHORTEST-PATHS}(R^{(m-1)}, W)$
return: R^3

Extended-Shortest-Paths(**Array** $R[n]$, **Matrix** $W[n \text{ x } n]$)
Matrix $H^1[n] = \{\infty, \cdots, \infty\}$ // Creo il nuovo vettore H di dimensione n pieno di ∞
for $i = 1$ **to** n **do**
  for $k = 1$ **to** n **do**
    $H[i] = \text{MIN}(H[i], R[k] + W[k, i])$
return: H

Complessità: Lavorando qui solo con il vettore corrispondente alla nostra sorgente il costo scende ad $O(n^2)$. Attenzione se esiste un ciclo di costo negativo di lunghezza al più 3 che include il vertice s, avremo un valore negativo sulla posizione $H(s, s)$.

Esercizio 1.2.2 #Floyd-Warshall

Sia $G = (V, E)$ un grafo i cui vertici sono identificati da 1 a $|V|$ e i cui archi sono pesati $w : E \to \mathbb{R}^+$. Trovare per ogni coppia di vertici in $V \times V$ il cammino di costo minimo che passa per i soli vertici identificati da un numero pari.

Esempio: è ammissibile il cammino da 1 a 7 che passa per i vertici 2 e 6, ossia 1-2-6-7, mentre non è ammissibile il cammino da 1 a 7 che passa per i vertici 2 e 3, ossia 1-2-3-7, perché 3 è un vertice identificato da un numero dispari.

- Descrivere a parole un algoritmo per risolvere l'esercizio;
- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto.

Elaborazione Soluzione: Poiché cerchiamo i cammini di costo minimo tra tutti i vertici di un grafo avente pesi positivi possiamo utilizzare l'algoritmo di Floyd-Warshall, che infatti lavora sui vertici. Si ricorda che per usare l'algoritmo di Floyd-Warshall si deve supporre che non vi siano cicli di costo negativo (Eventualmente trovabili con l'ausilio dell'algoritmo di Bellman-Ford). Essendo i pesi non negativi, non possono esistere cicli di costo negativo.

Qui la soluzione più immediata è una modifica dell'algoritmo di Floyd-Warshall, a fare in modo che esso consideri solo i vertici pari come vertici intermedi (dove $V = \{1, 2, \ldots, |V|\}$:

$$d_{u,v}^{(2k)} = \min\{d_{u,v}^{2(k-1)};\ d_{u,2k}^{2(k-1)} + d_{2k,v}^{2(k-1)}\}$$

L'equazione sopra riportata rappresenta l'equazione di ricorrenza per calcolare i cammini di costo minimo nel nostro problema (è una versione modificata di quella di Floyd-Warshall che tiene conto solo dei vertici pari '2k').

Inoltre definiamo:

- $D^{(0)}$: matrice delle adiacenze che rappresenta G modificato con ∞ se la coppia (i, j) non appartiene ad E.

$$D_{i,j}^{(0)} = \begin{cases} w(i,j) & \text{if } (i,j) \in E \text{ con } i \neq j \\ 0 & \text{if } i = j \\ +\infty & \text{if } (i,j) \notin E \end{cases}$$

- $\pi^{(0)}$: matrice delle adiacenze che rappresenta Π (cioè la matrice dei predecessori).

$$\pi_{i,j}^{(0)} = \begin{cases} \text{NULL} & \text{if } i = j \text{ or } (i,j) \notin E \\ i & \text{if } i \neq j \text{ and } (i,j) \in E \end{cases}$$

Pseudocodice:

Even-Floyd-Warshall(Grafo G)
// Abbiamo supposto di avere già definito $D_{i,j}^{(0)}$
for $k = 2$ **to** $\lfloor \frac{|V|}{2} \rfloor$ **step** 2 **do**
  for $i = 1$ **to** $|V|$ **do**
    for $j = 1$ **to** $|V|$ **do**
      $temp = D^{(k-2)}(i,k) + D^{(k-2)}(k,j)$
      if $D^{(k-2)}(i,j) > temp$ **then**
        $D^{(k)}(i,j) = temp$
        $\pi^{(k)}(i,j) = \pi^{(k-2)}(k,j)$ // Calcolo del padre
      else
        $D^{(k)}(i,j) = D^{(k-2)}(i,j)$
        $\pi^{(k)}(i,j) = \pi^{(k-2)}(i,j)$ // Calcolo padre per ricostruzione soluzione
return: $D^{\lfloor \frac{|V|}{2} \rfloor}$

Complessità:

- La complessità in tempo è $\frac{|V|}{2}|V|^2 = O(V^3)$
- La complessità in spazio è data dal fatto che si calcolano $\frac{|V|}{2}$ matrici per i costi dei cammini e $\frac{|V|}{2}$ matrici per tenere traccia dei cammini, tutte di dimensione $|V| \cdot |V|$; quindi abbiamo un costo totale di $O(V^3)$.

La ricostruzione dei cammini viene effettuata tramite l'algoritmo seguente, che viene evocato per ogni coppia di vertici di cui si vuole avere il cammino minimo che li connette:

ShortestPath(**Vertice** u, **Vertice** v, **Matrice Predecessori** π)
if $\pi[u, v] =$ NULL **then**
  return: $NoPaths$
List $path = \emptyset$
Node $x = v$ // Variabile locale per memorizzare i nodi
ENQUEUE($path, x$)
while $u \neq x$ **do**
  $x = \pi[u, x]$ // Percorriamo a Ritroso il Cammino Minimo tra u e v
  ENQUEUE($path, x$)
return: $path$

Esercizio 1.2.3 #Floyd-Warshall

Dato un grafo orientato e pesato $G = (V, E)$, i cui archi sono pesati con la funzione $w : E \to \mathbb{R}^+$ e un sotto-insieme di vertici $V' \subset V$, progettare un algoritmo per calcolare i costi dei cammini minimi fra tutte le coppie dei vertici di G che passano solo ed esclusivamente per i vertici in V'.
Dare lo pseudocodice dell'algoritmo e discutere la complessità in tempo dell'algoritmo.

Elaborazione Soluzione: Supponendo che il nostro grafo sia memorizzato con una matrice di adiacenza, $G[n \times n]$, possiamo usare l'algoritmo di Floyd-Warshall imponendo il passaggio solo per i vertici in V'.

Nell'algoritmo supponiamo che i vertici siano nominati con numeri interi da 1 ad n e che V sia quindi una lista di numeri che i cicli **for** scorrono uno alla volta.
Supponiamo anche che $G[n \times n]$ sia come segue:

$$G_{i,j} = \begin{cases} w(i,j) & \text{if } (i,j) \in E \text{ con } i \neq j \\ 0 & \text{if } i = j \\ +\infty & \text{if } (i,j) \notin E \end{cases}$$

Pseudocodice:

Floyd-Warshall-V'(**Grafo** $G[n \times n]$)
$D^{(0)}_{i,j} = G[n \times n]$
for $k \in V'$ **do**
  for $i \in V$ **do**
    for $j \in V$ **do**
      // L'esercizio ci richiede solo i costi dei cammini
      $D^{(k)}(i,j) =$ MIN($D^{\text{PREVIOUS } k}(i,j), D^{\text{PREVIOUS } k}(i,k) + D^{\text{PREVIOUS } k}(k,j)$)
return: $D^{(n)}$

Complessità: L'algoritmo di Floyd-Warshall impiega $O(|V|^3)$, qui però il primo **for** non scorre tutti e $|V|$ i vertici ma $|V'|$, quindi il costo diviene: $O(|V'| \cdot |V|^2)$.

Esercizio 1.2.4 #MetodoTropicale

Dato un intero k, con $1 \leq k \leq \lceil log_2 |V| \rceil$, e un grafo orientato G, i cui archi sono pesati con la funzione $w : E \to \mathbb{R}^+$, descrivere un algoritmo per trovare per ciascuna coppia di nodi il cammino di lunghezza minima che attraversa al più k archi.
Discutere la complessità in tempo della soluzione proposta.

Elaborazione Soluzione: L'unico algoritmo che conosciamo che lavora con gli archi e che funziona su grafi senza cicli di costo negativo è il Metodo Tropicale, anche detto algoritmo APSP (All Pairs Shortest Paths).
In particolare ci interessa che l'algoritmo generi la matrice k-esima, per cui va leggermente modificato.

Pseudocodice:

```
Fast-Tropical-Method-k(Matrix W[n × n], int k) // W matrice di adiacenza con pesi
Matrix R[n × n] = W
int m = 1
// Cerchiamo fino alla k-esima matrice
while m ≤ k − 1 do
    R^{2m} = Extended-Shortest-Paths(R^m, R^m)
    m = 2 · m
return: R^k

Extended-Shortest-Paths(Matrix R[n × n], Matrix W[n × n])
Matrix H[n × n] = {∞ ··· ∞} // Creiamo una matrice piana di +∞
for i = 1 to n do
    for j = 1 to n do
        for h = 1 to n do
            H[i, j] = min(H[i, j], R[i, h] + W[h, j])
return: H
```

Complessità: L'algoritmo ha costo $O(\log k \cdot n^3)$; nel caso peggiore, la descrizione del problema ci dice che k ha valore massimo $\log n$, quindi il costo nel caso peggiore è:

$$O((\log \log n) \cdot n^3)$$

.

Esercizio 1.2.5 #Johnson

Dato un grafo $G = (V, E)$, orientato e pesato, trovare i cammini minimi tra tutte le coppie con al più costo k.

- Descrivere l'algoritmo che si propone
- Darne la descrizione in pseudocodice
- Discuterne la complessità in tempo dell'algoritmo.

Elaborazione Soluzione: Poiché abbiamo pesi qualsiasi, potremo avere cicli di costo negativo. Potremo quindi prima fare un esecuzione di Bellman-Ford per vedere se vi sono e poi un Metodo Tropicale fino alla k-esima matrice, ma ciò avrebbe costo $O(E \cdot V) + O(\log k \cdot |V|^3)$. Possiamo invece applicare l'algoritmo di Johnson, che utilizza di per sé l'algoritmo di Bellman-Ford come subroutine, ma, va modificato.

Funzionamento dell'algoritmo di Johnson: Questo per prima cosa aggiunge una "super sorgente" S al grafo G originale, connettendola ad ogni altro vertice del grafo con un arco di peso 0. L'algoritmo lavora tipicamente con liste di adiacenza. Fatto ciò, si esegue sul nuovo grafo così ottenuto, G', l'algoritmo di Bellman-Ford, con sorgente la super sorgente S. Questo permette di verificare se vi sono o no cicli di costo negativo, in caso affermativo il tutto termina. Se non vi sono cicli di costo negativo, con i risultati dell'algoritmo di Bellman-Ford, quindi i cammini minimi dalla super sorgente S ad ogni vertice del grafo, viene modificato il peso degli archi del grafo G'. Con la formula seguente: $\omega'(u, v) = \omega(u, v) + h(u) - h(v)$; dove $\omega'(u, v)$ è il nuovo peso dell'arco (u, v), $\omega(u, v)$ il peso originale ed $h(u)$ e $h(v)$ sono rispettivamente il cammino minimo da S ad u e da S ad v calcolati dall'algoritmo di Bellman-Ford.
Infine, si toglie la super sorgente S dal grafo G' (nell'implementazione per risparmiare tempo si lavora sul grafo originale G usando la funzione dei pesi di G', ω') e per ogni nodo del grafo si esegue l'algoritmo di Dijkstra.
La distanza nel grafo originale è poi calcolata aggiungendo al risultato dato dall'algoritmo di Dijkstra $h(v) - h(u)$; il tutto è memorizzato in una matrice.

Quindi: Essenzialmente quello che va modificato è l'algoritmo di Dijkstra, che per risolvere ottimamente il nostro problema deve fermarsi quando estrae dall'heap un vertice a distanza maggiore di k.

Pseudocodice:

```
Johnson(Grafo G, Funzione Peso ω, Int k)
// Supponiamo il grafo memorizzato tramite liste di adiacenza
// Aggiunta della super sorgente S al grafo
G'.V = G.V ∪ {S}
G'.E = G.E ∪ {(S, v) | v ∈ G.V}
ω(S, v) = 0 ∀ v ∈ G.V
if Bellman-Ford(G', ω, S) == False then
    return: null
else
    // h(x) | x ∈ G.V è la distanza computata da Bellman-Ford
    for ogni arco (u, v) ∈ G'.E do
        ω'(u, v) = ω(u, v) + h(u) − h(v)
    Matrice D[n × n] = ∅
    for ogni vertice u ∈ G.V do
        // g(u, v) è la distanza tra u ed ogni vertice v computata da Dijkstra-K
        g(u, v) | v ∈ G.V = Dijkstra-K(G, ω', u, k)
        for ogni vertice v ∈ G.V do
            D[u, v] = g(u, v) + h(v) − h(u)
    return: D

Dijkstra-K(Grafo G, Funzione Peso ω, Sorgente S, Int k)
for ogni vertice v ∈ G.V do
    v.d = +∞
    v.predecessore = null
S.d = 0
Lista L = ∅
Min-Heap Q = ∅
for ogni vertice v ∈ G.V do
    insert(Q, v)
stop = False // Variabile per fermare il ciclo while
while is-empty(Q) == False and stop == True do
    u = extractMin(Q)
    // Rilassiamo gli archi connessi ad u fino a stima k
    if u.d > k then
        stop = True
        // Usciamo dal ciclo appena troviamo un vertice a distanza maggiore di k
    else
        for ogni vertice v ∈ G.adj[u] do
            if v.d > u.d + ω(u, v) then
                v.d = u.d + ω(u, v)
                v.predecessore = u
    enqueue(L, u)
```

Complessità: La complessità è quella dell'algoritmo di Johnson, $O(V \cdot E \cdot \log V)$, dove $O(E \cdot \log V)$ è dato dall'algoritmo di Dijkstra. Dijkstra-K si ferma quando è estratto un nodo a distanza maggiore di k, che però essendo una distanza non ci permette di determinare il costo in sua funzione, non avendo particolari informazioni sulla struttura del grafo da correlarvi.

Esercizio 1.2.6 #Floyd-Warshall

Dato un grafo G orientato e pesato, trovare i cammini di costo minimo fra tutte le coppie di vertici in G.

- Descrivere l'algoritmo
- Dare lo pseudocodice dell'algoritmo
- Studiare la complessità dell'algoritmo proposto

Elaborazione Soluzione: Possiamo applicare l'algoritmo Floyd-Warshall per la ricerca dei cammini

minimi tra tutte le coppie.

Pseudocodice:

Floyd-Warshall(**Grafo** G)
int $D^0[n \times n] = G.mat$
for $k = 1$ **to** $G.V$ **do**
  int $D^k[n \times n] = \{0 \cdots 0\}$
  for $i = 1$ **to** $|G.V|$ **do**
    for $j = 1$ **to** $|G.V|$ **do**
      $D^k[i,j] = \text{MIN}(D^{k-1}[i,j], (D^{k-1}[i,k] + D^{k-1}[k,j]))$
return: D^n

Complessità: Questo ha costo $O(V^3)$.